Sprint 1 - Carnet du Produit

Auteurs : L3 CMI Informatique

- Angelo Adragna
- Corentin Saccoman
- Elliot Pozucek
- Colin Palazzetti Rubera
- Elvin Fichot-Corgeron

Rôles

- Angelo est le maître de `SCRUM`.
- Corentin et Elliot travailleront sur la méthode `pdftotext`, ensemble dans un seul fichier en Python.
- Colin et Elvin travailleront sur la méthode `pdf2txt`, chacun de leur côté : Colin dans un fichier `.py` et Elvin dans un autre.

Objectif

Fournir une évaluation qualitative (manuellement par inspection) des sorties afin de décider quels outils et options utiliser :

- Frontières des phrases : Vérifiez si les phrases sont correctement segmentées.
- écoupage des mots : Assurez-vous que les mots ne sont pas incorrectement séparés, notamment aux endroits où il y a des coupures de ligne ou des traits d'union.
- Alignement des lignes ou des phrases : Comparez les sorties avec le fichier original pour repérer des cas où les lignes ou les phrases sont mélangées, dédoublées ou manquantes.
- Traitement des tableaux : Vérifiez que les données des tableaux sont bien structurées, avec les relations entre les colonnes et les lignes préservées.
- Caractères spéciaux et symboles : Assurez-vous que les caractères spéciaux sont correctement interprétés et non transformés en caractères invalides.
- Hiérarchie et formatage : Vérifiez la hiérarchie des titres, sous-titres, listes, et autres éléments formatés.
- Langue et sens : Inspectez si la langue et le sens des textes sont respectés sans distorsion.
- Métadonnées : Assurez-vous que les métadonnées pertinentes du fichier source sont extraites correctement.
- Compatibilité des outils : Testez différents outils pour identifier ceux qui offrent les meilleurs résultats pour vos besoins spécifiques.
- etc.

Nous réussir à extraire correctement :

- Les titres

- Les auteurs
- Un abstract
- Une introduction
- Un corps
- Une conclusion
- Une discussion
- Une bibliographie

Veuillez apporter une attention particulière à la façon dont ces différents éléments sont agencés afin de réussir à vous adapter au plus de cas possibles !

Travail de chaque codeur pendant la mêlée (Revue Sprint)

Consignes : Déposez vos comptes rendu au plus tard le 04/04/2025 sur la branche 1

---

1. **Corentin et Elliot** : `pdftotext`
   - **Options de lancement** :
     - `pdftotext -H 20 | 1080`
       - **Avantages** : Aucun.
       - **Inconvénients** : Produit un fichier texte vide.
     - `pdftotext -x 0 | 1 | 640 | 1200000 | 12000`
       - **Avantages** : Aucun.
       - **Inconvénients** : Produit un fichier texte vide.
     - `pdftotext -y 0 | 1 | 640 | 1200000 | 12000`
       - **Avantages** : Aucun.
       - **Inconvénients** : Produit un fichier texte vide.
     - `pdftotext -colspacing 0.1 | 0.3 | 0.7`
       - **Avantages** : Aucun.
       - **Inconvénients** : N'affecte pas le fichier texte par rapport à la commande sans option.
     - `pdftotext -eol unix | mac`
       - **Avantages** : Aucun.
       - **Inconvénients** : N'affecte pas le fichier texte par rapport à la commande sans option.
     - `pdftotext -nopgbrk`
       - **Avantages** : Aucun.

    - **Inconvénients** : N'affecte pas le fichier texte par rapport à la commande sans option.

  - `pdftotext -r 320 | 460 | 640 | 3 | 72`
    - **Avantages** : Aucun.
    - **Inconvénients** : N'affecte pas le fichier texte par rapport à la commande sans option.

  - `pdftotext -raw`
    - **Avantages** :
      - Fonctionne bien pour les structures en colonnes : le texte reste cohérent et l'ordre est respecté.
      - Les tableaux conservent une structure cohérente.
    - **Inconvénients** :
      - Réduction des espaces, rendant les tableaux illisibles pour les systèmes et peu lisibles pour les humains.
      - Les figures deviennent linéaires et incompréhensibles.
      - Les mots coupés en deux dans le PDF restent séparés dans le fichier texte.
      - Les changements de pages sont visibles et interrompent le texte.

  - `pdftotext -fixed 5`
    - **Avantages** :
      - Conserve (en partie) la structure du PDF.
      - Paragraphes bien visibles et séparés.
      - Certaines expressions mathématiques sont bien retranscrites.
      - Les tableaux sont très bien retranscrits.
      - Les diagrammes sont reconnaissables (mais illisibles).
    - **Inconvénients** :
      - Catastrophique pour les colonnes : le texte est mélangé et illisible.
      - Expressions mathématiques complexes mal retranscrites.
      - Changements de pages visibles, interrompant le texte.
      - Mots coupés à la ligne également coupés dans le fichier texte.

  - `pdftotext -layout`
    - **Avantages** :
      - Permet de garder la mise en forme, utile pour les tableaux.
    - **Inconvénients** :
      - La mise en forme colonne par colonne est conservée, ce qui peut être indésirable dans certains cas.

- **Appréciation générale** :

pdftotext est très simple d'utilisation mais son utilisation sur des articles scientifiques ayant un format assez particulier, avec des graphiques, des tableaux et des colonnes nécessite un énorme travail en post-traitement (script python ou bash) pour récupérer le bon format de texte et bien structurer le document (Parties Auteur, titre…).

- **Note** :
  Nous n'avons pas explorer la bonne piste, car la consigne demande ne demande pas d'avoir une retranscription parfaite du pdf, mais d'avoir un bon résumé, lisible et facile à comprendre.
  Nous devons donc concentrer nos efforts sur la réalisation d'un résumé efficace, et ce peu importe le format du pdf (colonne, tableau, figure, etc.). À voir avec le professeur pour des précisions sur la longueur du résumé.

---

2. **Colin** : `pdf2txt`
   - **Options de lancement** :
     - `pdf2txt -t xml Word2Vec.pdf > output.html`
       - **Avantages** :
         - Conserve la structure des blocs de texte avec la balise `textbox`.
         - Conserve les espaces, même en fin de ligne avec `<text> </text>`.
       - **Inconvénients** :
         - Présence de `<text>\n</text>` à toutes les fins de lignes.
         - Perte de la structure des tableaux.
     - `pdf2txt -t html Word2Vec.pdf > output.html`
       - **Avantages** :
         - Affiche un rendu intéressant à observer.
       - **Inconvénients** :
         - Structure étrange et incohérente.
         - Les tableaux sont difficiles à lire et apparaissent parfois par-dessus le texte.

   Python : `PyPDF2`
   - **Options de lancement** :
     - Utilisation de la bibliothèque PyPDF2 avec un script personnalisé pour extraire le texte.
   - **Exemple de code** :

```
html_file = open('CORPUS_TRAIN/Word2Vec.html', 'w')

from PyPDF2 import PdfReader

parts = []

def visitor_body(text, cm, tm, fontDict, fontSize):
    y = tm[5]
    if y > 50 and y < 720:
        parts.append(text)

reader = PdfReader('CORPUS_TRAIN/Word2Vec.pdf')
page = reader.pages[0]
extracted_text = page.extract_text(0, visitor_text=visitor_body)

html_file.write(extracted_text)
html_file.close()
```

- **Avantages** :
  - Extraction du texte basée sur une analyse précise des coordonnées.
  - Permet une personnalisation du traitement des données extraites.
- **Inconvénients** :
  - Les résultats globaux en termes de structure sont parfois meilleurs avec les commandes bash.
  - Nécessite une adaptation et des connaissances en Python pour un traitement optimal.

- **Appréciation globale** : `pdf2txt` propose des options intéressantes pour l'extraction de texte, notamment en XML et HTML. L'option en XML est utile pour conserver une structure textuelle précise, tandis que l'option en HTML permet un aperçu unique bien que parfois incohérent. Cependant, la perte de la structure des tableaux est un inconvénient notable. Quant à PyPDF2, son approche personnalisée est prometteuse pour des extractions spécifiques, mais elle demande un investissement en termes de compétences et de traitements supplémentaires. Une combinaison de ces outils pourrait être envisagée selon les besoins spécifiques, en tenant compte de leurs avantages et inconvénients respectifs.

---

3. **Elvin** : `pdf2txt`
   - **Options de lancement** :
     - `pdf2text Torres.pdf`

    - **Avantages** :
      - Permet d'afficher directement le contenu du fichier dans le terminal.
    - **Inconvénients** :
      - Limité à une visualisation terminale sans sauvegarde structurée.
      - Les formules mathématiques et les tableaux ne sont pas bien parsés.
      - Certains symboles, comme le signe de somme, ne sont pas traités.

  - `pdf2text Torres.pdf > Torres.txt`
    - **Avantages** :
      - Création d'un fichier texte lisible.
    - **Inconvénients** :
      - Mauvais parsage des formules mathématiques et des tableaux.
      - Certains symboles spécifiques ne sont pas reconnus (exemple : somme).
      - Manque de précision dans la structure des données.

  - `pdf2txt -t xml Torres.pdf > Torres.xml`
    - **Avantages** :
      - Met chaque caractère dans une balise `<text>`, permettant une granularité fine.
      - Chaque ligne est contenue dans une balise `textline`, utile pour regrouper les données des tableaux.
      - Les colonnes sont bien identifiées grâce aux balises `textbox`, avec des IDs indiquant leur ordre.
    - **Inconvénients** :
      - Chaque espace est représenté par une balise vide, rendant le fichier complexe.
      - Mauvais regroupement des caractères pour les formules mathématiques.
      - Les signes `<` ne sont pas pris en compte, contrairement au format `.txt`.

- **Solution suggérée** :
  - Pour les tableaux : transformer le PDF en XML, puis extraire les tableaux via les balises.
  - Pour les caractères : utiliser l'option `-c` et préciser les paramètres nécessaires.
  - Pour les formules mathématiques : convertir en XML, puis repérer les symboles particuliers en utilisant un script Python avec `pdfminer`.

- **Appréciation globale** : `pdf2txt` offre des outils efficaces pour l'extraction de données, notamment en XML, grâce à ses balises structurées. Cependant, son traitement des formules mathématiques, des tableaux et de certains symboles spécifiques reste limité. Des solutions mixtes, comme l'utilisation de scripts Python pour compléter le travail, sont recommandées pour répondre aux besoins complexes tels que les articles scientifiques. Malgré ses inconvénients, `pdf2txt` constitue une base intéressante pour un traitement approfondi des PDF avec une approche personnalisée.

–

Retour sur le travail effectué (Rétrospective Sprint) : Quels outils et options utiliser ?

Ce qui a bien fonctionné

- L'utilisation de `pdftotext` avec l'option `-raw` ou `-layout` a permis d'obtenir une structure globale relativement fidèle, notamment pour les colonnes et les tableaux simples.
- L'option `-fixed 5` a révélé un bon rendu pour les paragraphes et certaines expressions mathématiques.
- L'export XML via `pdf2txt -t xml` s'est avéré très utile pour localiser et segmenter les blocs de texte avec précision grâce aux balises `textbox` et `textline`.
- L'outil **PyPDF2**, bien que nécessitant un développement personnalisé, a permis une extraction fine basée sur des coordonnées précises, ouvrant la voie à des traitements ciblés.
- La collaboration en binôme a permis un partage efficace des tests et une exploration plus large des possibilités.

Ce qui a posé problème

- De nombreuses options testées avec `pdftotext` (`-x`, `-y`, `-r`, `-eol`, `-colspacing`, etc.) n'ont produit aucun résultat ou un fichier vide, ce qui a ralenti les expérimentations.
- `pdftotext`, bien qu'efficace dans certains cas, exige un post-traitement important pour corriger les erreurs d'alignement, les coupures de mots, et la segmentation des phrases.
- `pdf2txt`, malgré une bonne structuration XML, complique parfois l'analyse à cause de sa granularité excessive (chaque caractère dans une balise).
- La perte de structure des tableaux et la mauvaise gestion des symboles mathématiques restent un obstacle majeur.
- Le travail a initialement été trop centré sur la fidélité à la structure originale, au lieu de se concentrer sur la production d'un résumé clair et lisible, comme l'indiquait l'objectif.

Ce qu'on doit améliorer

- Cibler dès le début du second Sprint l'objectif exact, afin d'optimiser le temps de travail.
- Se recentrer sur l'objectif principal : **produire un résumé compréhensible et exploitable**, et non une copie exacte du PDF.
- Utiliser les outils pour **extraire les grandes sections** (titre, auteurs, abstract, introduction, etc.) plutôt que tout le contenu brut.
- Envisager une **combinaison des outils** : par exemple `pdf2txt` pour structurer et localiser, puis un script Python pour nettoyer et synthétiser.
- Continuer à explorer les **scripts personnalisés en Python**, qui offrent une meilleure souplesse et permettent de s'adapter aux différents types de documents.
- Clarifier avec l'enseignant les **attentes précises sur le niveau de détail du résumé**, afin d'ajuster la granularité de l'extraction.

Conclusion : Quel outil pour quel besoin ?

- Pour **un rendu brut rapide**, utilisable directement ou à nettoyer ensuite → `pdftotext -layout` **ou** `-raw`
- Pour **un traitement structuré, précis et adaptable avec un script Python** → `pdf2txt -t xml`
- Pour **reconnaître les zones de texte** (titre, auteurs, sections) de manière automatique → `pdf2txt` **avec parsing XML**
- Pour **résumer un document** (objectif principal du projet) → **combinaison de** `pdf2txt` **+ script Python** pour structurer + extraire ce qui est pertinent

Nous allons donc utiliser une combinaison de `pdf2txt` `pdftotext` et de `python`.